

Git Machine: Technical Overview

Implementation: github.com/GitM-Labs/runtime Benchmarks: [benchmark README](#)

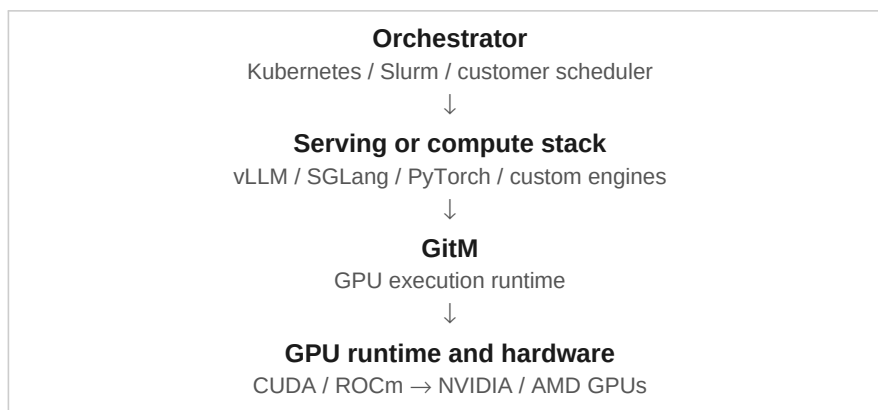
Shared for evaluation. Please keep within your team.

Summary

Compute providers can add capacity by deploying more hardware or by increasing the useful work completed on the hardware they already operate. Git Machine (GitM) is a GPU execution runtime that helps inference and compute providers increase the capacity of existing GPUs.

GitM reconstructs how a workload is executing from hardware telemetry, models what it should be capable of achieving on the deployed system, and identifies the runtime behavior responsible for the gap. It then applies and verifies targeted changes to scheduling, concurrency, memory movement, synchronization, and communication.

Where does GitM sit?



GitM sits between the serving or compute stack and the GPU runtime. The existing stack continues to control workload placement, requests, batching, model-serving state, and kernel selection, while GitM operates on the execution that stack produces.

What does GitM touch, and what can it see?

Getting into that position requires no changes to the rest of the stack. GitM runs in user space and integrates through supported runtime or plugin interfaces. In Kubernetes environments, an operator manages deployment, canary rollout, and rollback, while a lightweight per-node component captures GPU and execution telemetry. When a supported framework integration is unavailable, GitM can attach at the GPU runtime layer, where kernel launches, memory operations, streams, and synchronization remain observable.

The system requires no GPU driver replacement or kernel module. It operates from execution telemetry without ingesting model weights, source code, architecture files, prompts, or outputs, and the collected telemetry can remain within the customer's environment.

How does GitM work?

From that telemetry, GitM runs a continuous four-stage loop: model expected performance, attribute the performance gap, apply a targeted runtime change, and verify the result.

1. Model expected performance

GitM reconstructs the workload's execution structure from runtime telemetry and combines it with measured characteristics of the GPU, memory system, interconnect, and cluster topology. This produces a workload-specific execution ceiling that estimates what the complete job should be capable of achieving on the system where it is running. Roofline analysis establishes the workload's compute and memory limits, and GitM's execution model also accounts for dependencies, launch gaps, synchronization, communication, and missed overlap. This job-level view is necessary because efficient individual kernels can still produce poor end-to-end performance when the execution around them is inefficient. What comes out of this stage is a measured gap between what the workload did and what it should have been able to do:

2. Attribute the performance gap

GitM compares observed execution with the modeled ceiling, evaluating compute utilization, concurrency, synchronization, memory movement, and communication, and attributes significant losses to their likely runtime causes. The output of this stage is an attributed cause tied to evidence, with an estimate of how much performance is recoverable.

3. Apply a targeted runtime change

Once GitM has attributed a performance loss, it selects a runtime change matched to the diagnosed cause. For example, a workload losing time to launch gaps may have its repeated kernel sequences captured into one graph launch; one whose transfers block compute may get new stream assignment and earlier prefetch. The intervention surface includes:

Area	Available changes
Launch behavior	Capture repeated kernel and memory-operation sequences into graph launches, reduce host-side launch overhead, and adjust launch configuration exposed by the runtime.
Concurrency and scheduling	Change stream assignment and priorities, increase safe overlap between independent work, and reduce unnecessary synchronization.
Memory movement	Improve transfer and compute overlap, prefetch timing, allocation behavior, and runtime memory management.
Communication	Tune collective configuration and communication behavior for the deployed cluster topology.
Runtime configuration	Use live measurement to choose among equivalent implementations and adjust serving-engine or execution parameters exposed by the existing stack.

GitM does not write or rewrite kernels. Better kernels improve the starting point; GitM improves how those kernels execute together.

4. Verify the result

Each change is measured against the original workload and retained only when the result confirms an improvement without violating latency, memory, stability, or correctness requirements. Changes can be introduced on a controlled portion of execution and automatically reverted on regression, and output equivalence can be verified when required. The system records the diagnosed loss, the intervention applied, and the measured result, so each accepted optimization has a clear causal basis.

How is GitM different from what I already run?

GitM complements the systems it is most often compared to rather than replacing any of them:

System	Primary role	Relationship to GitM
Compilers	Generate execution plans	GitM optimizes the resulting execution using live workload behavior
Kernel systems	Improve individual operations	GitM optimizes execution around and between those operations
Serving engines	Manage requests, batching, and model-serving state	GitM optimizes the hardware execution produced by those policies
Profilers	Record and display execution behavior	GitM identifies recoverable loss, applies changes, and verifies the result
Orchestrators	Place workloads across infrastructure	GitM optimizes execution after placement

Results

On GitM's published mechanism-validation benchmarks, the loop recovers **32.4% higher throughput with byte-identical output on HFT** and **63% higher throughput on KITTI**, behind the gates and rollback semantics described above. Full methodology and reproduction instructions are available in the [benchmark README](#).

GitM automates the performance engineering required to close the gap between the execution a workload produces and what its deployed hardware should achieve.